

Decision_Tree_Classifier

August 13, 2025

1 Decision Tree Classifier Practical Implementation

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')
```

```
[2]: df = sns.load_dataset('iris')
df.head()
```

```
[2]:   sepal_length  sepal_width  petal_length  petal_width  species
0           5.1           3.5           1.4           0.2   setosa
1           4.9           3.0           1.4           0.2   setosa
2           4.7           3.2           1.3           0.2   setosa
3           4.6           3.1           1.5           0.2   setosa
4           5.0           3.6           1.4           0.2   setosa
```

```
[3]: df['species'].value_counts()
```

```
[3]: species
setosa      50
versicolor  50
virginica   50
Name: count, dtype: int64
```

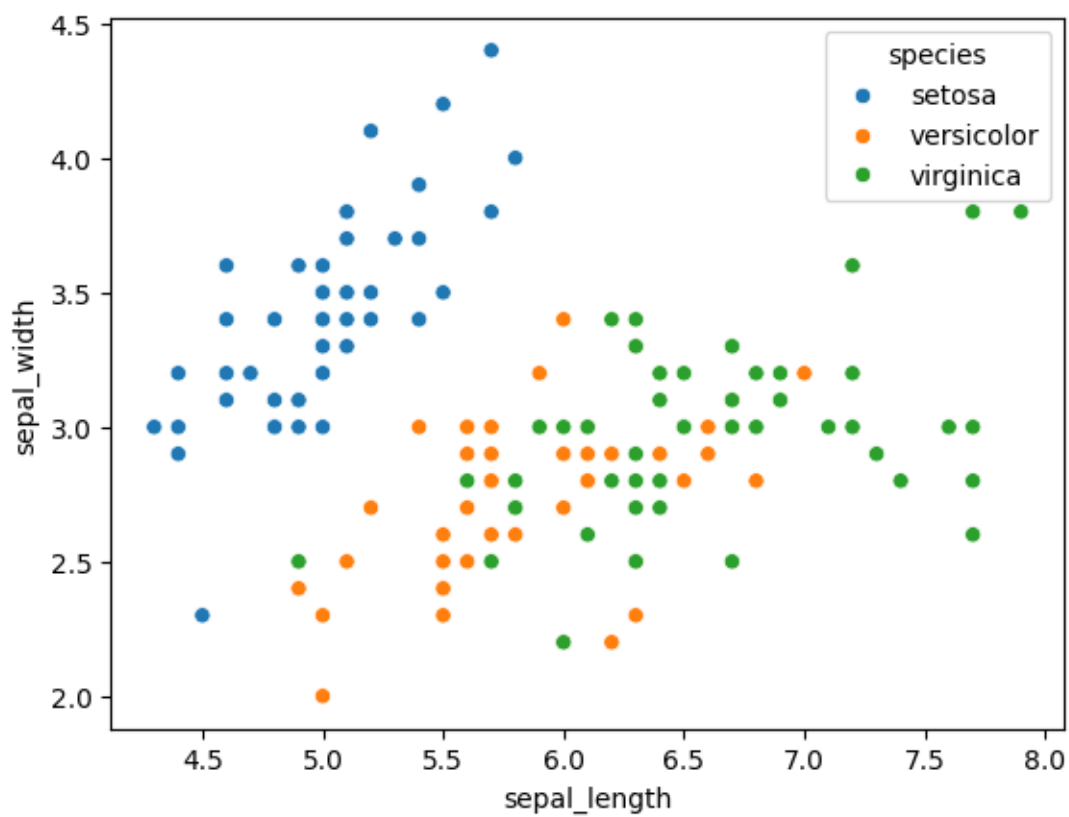
```
[4]: df.columns
```

```
[4]: Index(['sepal_length', 'sepal_width', 'petal_length', 'petal_width',
        'species'],
        dtype='object')
```

```
[5]: # fig, axes = plt.Subplots(2,2, figsize(8,6))
```

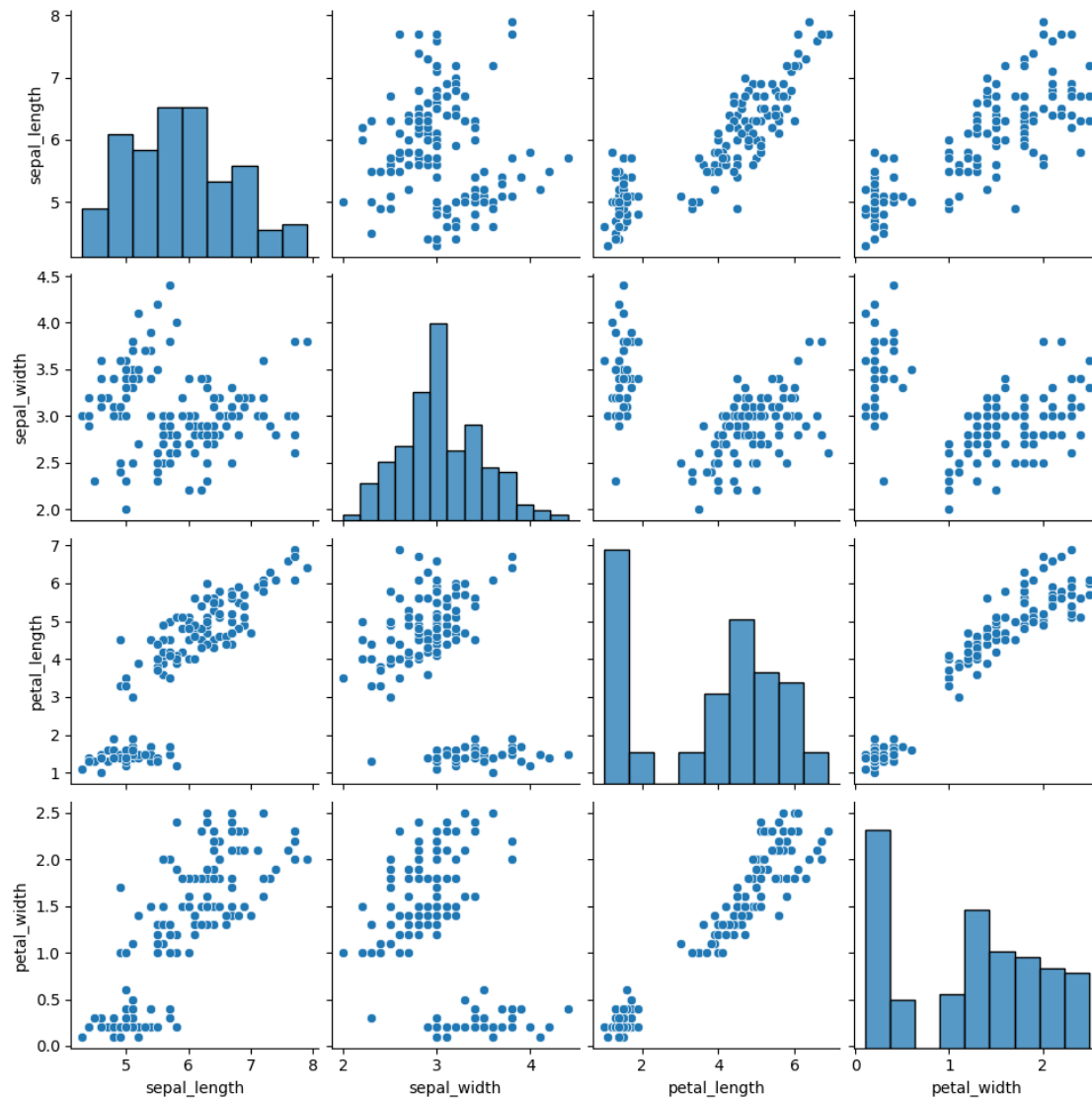
```
[6]: sns.scatterplot(x = df['sepal_length'], y = df['sepal_width'],
                    ↪hue=df['species'])
```

```
[6]: <Axes: xlabel='sepal_length', ylabel='sepal_width'>
```



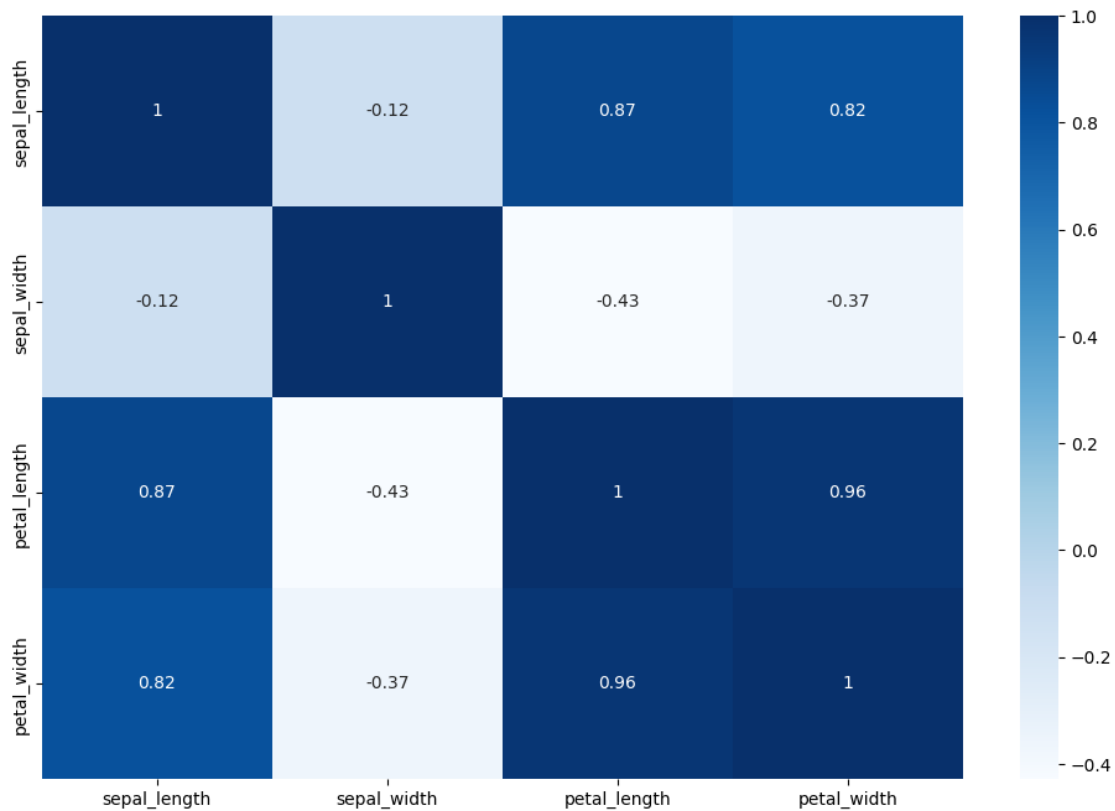
```
[7]: sns.pairplot(df)
```

```
[7]: <seaborn.axisgrid.PairGrid at 0x19dc50c8070>
```



```
[8]: plt.figure(figsize=(12,8))
      sns.heatmap(df.drop('species', axis=1).corr(), annot=True, cmap='Blues')
```

```
[8]: <Axes: >
```



```
[9]: from sklearn.datasets import load_iris
iris = load_iris()
```

```
[10]: y = iris['target']
X = df.drop('species', axis=1)
```

```
[11]: X.head()
```

```
[11]:
```

	sepal_length	sepal_width	petal_length	petal_width
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

```
[12]: from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
↳ random_state=10)
```

```
[13]: ## Apply decision tree classifier
```

```
from sklearn.tree import DecisionTreeClassifier
treeclsi = DecisionTreeClassifier()
```

```
[14]: treeclsi.fit(X_train, y_train)
```

```
[14]: DecisionTreeClassifier()
```

```
[15]: from sklearn import tree
plt.figure(figsize=(18,18))
tree.plot_tree(treeclsi, filled=True)
```

```
[15]: [Text(0.3076923076923077, 0.9285714285714286, 'x[3] <= 0.8\ngini =
0.665\nsamples = 120\nvalue = [40, 37, 43]'),
Text(0.23076923076923078, 0.7857142857142857, 'gini = 0.0\nsamples = 40\nvalue
= [40, 0, 0]'),
Text(0.2692307692307693, 0.8571428571428572, 'True '),
Text(0.38461538461538464, 0.7857142857142857, 'x[2] <= 4.75\ngini =
0.497\nsamples = 80\nvalue = [0, 37, 43]'),
Text(0.34615384615384615, 0.8571428571428572, ' False'),
Text(0.15384615384615385, 0.6428571428571429, 'x[3] <= 1.65\ngini =
0.057\nsamples = 34\nvalue = [0, 33, 1]'),
Text(0.07692307692307693, 0.5, 'gini = 0.0\nsamples = 33\nvalue = [0, 33, 0]'),
Text(0.23076923076923078, 0.5, 'gini = 0.0\nsamples = 1\nvalue = [0, 0, 1]'),
Text(0.6153846153846154, 0.6428571428571429, 'x[3] <= 1.75\ngini =
0.159\nsamples = 46\nvalue = [0, 4, 42]'),
Text(0.38461538461538464, 0.5, 'x[1] <= 2.65\ngini = 0.49\nsamples = 7\nvalue =
[0, 3, 4]'),
Text(0.3076923076923077, 0.35714285714285715, 'gini = 0.0\nsamples = 2\nvalue =
[0, 0, 2]'),
Text(0.46153846153846156, 0.35714285714285715, 'x[2] <= 5.05\ngini =
0.48\nsamples = 5\nvalue = [0, 3, 2]'),
Text(0.38461538461538464, 0.21428571428571427, 'gini = 0.0\nsamples = 2\nvalue
= [0, 2, 0]'),
Text(0.5384615384615384, 0.21428571428571427, 'x[1] <= 2.75\ngini =
0.444\nsamples = 3\nvalue = [0, 1, 2]'),
Text(0.46153846153846156, 0.07142857142857142, 'gini = 0.0\nsamples = 1\nvalue
= [0, 1, 0]'),
Text(0.6153846153846154, 0.07142857142857142, 'gini = 0.0\nsamples = 2\nvalue =
[0, 0, 2]'),
Text(0.8461538461538461, 0.5, 'x[2] <= 4.85\ngini = 0.05\nsamples = 39\nvalue =
[0, 1, 38]'),
Text(0.7692307692307693, 0.35714285714285715, 'x[0] <= 5.95\ngini =
0.5\nsamples = 2\nvalue = [0, 1, 1]'),
Text(0.6923076923076923, 0.21428571428571427, 'gini = 0.0\nsamples = 1\nvalue =
[0, 1, 0]'),
Text(0.8461538461538461, 0.21428571428571427, 'gini = 0.0\nsamples = 1\nvalue =
[0, 0, 1]'),
```

```
Text(0.9230769230769231, 0.35714285714285715, 'gini = 0.0\nsamples = 37\nvalue = [0, 0, 37]')]
```



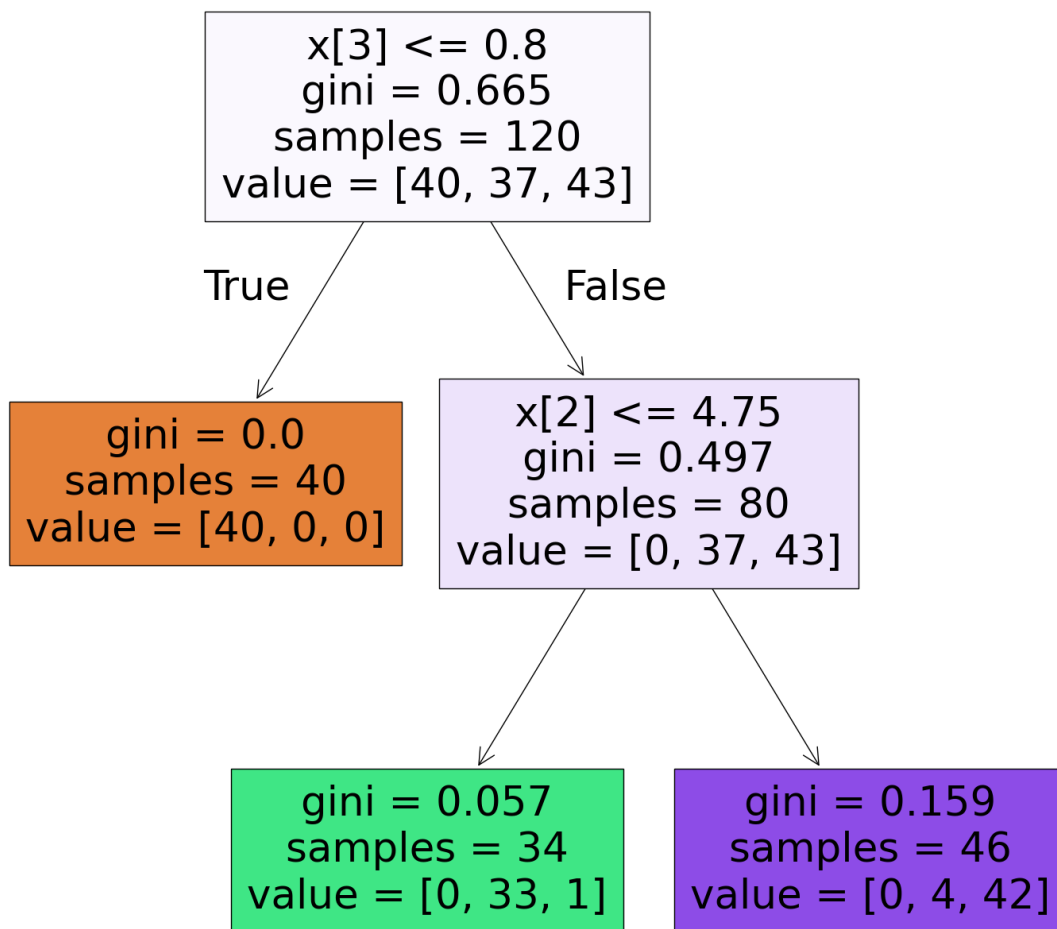
Post pruning to prevent the overfitting so we use the `max_depth` and other to prune the tree to reducing the depth of the tree

```
[16]: model = DecisionTreeClassifier(max_depth=2)
      model.fit(X_train, y_train)
```

```
[16]: DecisionTreeClassifier(max_depth=2)
```

```
[17]: plt.figure(figsize=(18,18))
      tree.plot_tree(model, filled=True)
```

```
[17]: [Text(0.4, 0.8333333333333334, 'x[3] <= 0.8\ngini = 0.665\nsamples = 120\nvalue = [40, 37, 43]'),
Text(0.2, 0.5, 'gini = 0.0\nsamples = 40\nvalue = [40, 0, 0]'),
Text(0.30000000000000004, 0.6666666666666667, 'True '),
Text(0.6, 0.5, 'x[2] <= 4.75\ngini = 0.497\nsamples = 80\nvalue = [0, 37, 43]'),
Text(0.5, 0.6666666666666667, ' False'),
Text(0.4, 0.16666666666666666, 'gini = 0.057\nsamples = 34\nvalue = [0, 33, 1]'),
Text(0.8, 0.16666666666666666, 'gini = 0.159\nsamples = 46\nvalue = [0, 4, 42]')] ]
```

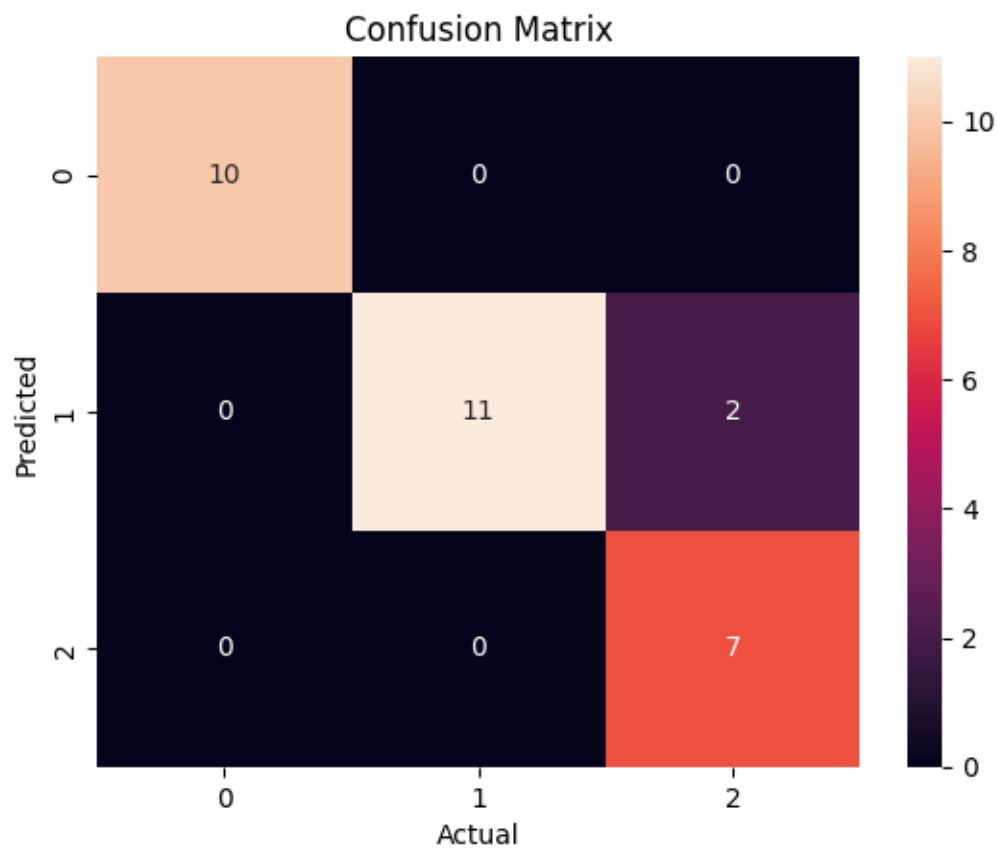


here you can see that it is going to only the depth of 2

```
[18]: y_pred = model.predict(X_test)
```

```
[19]: from sklearn.metrics import accuracy_score, confusion_matrix, \
      ↪classification_report
      sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt='d')
      plt.xlabel('Actual')
      plt.ylabel('Predicted')
      plt.title('Confusion Matrix')
```

```
[19]: Text(0.5, 1.0, 'Confusion Matrix')
```



```
[20]: print(accuracy_score(y_test, y_pred))
```

```
0.9333333333333333
```

```
[21]: print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	0.85	0.92	13

	2	0.78	1.00	0.88	7
accuracy				0.93	30
macro avg	0.93	0.95	0.93		30
weighted avg	0.95	0.93	0.93		30

2 Decision Tree Classifier Prepruning And Hyperparameter Tuning

```
[22]: params = {
    'criterion':['gini', 'entropy', 'log_loss'],
    'splitter':['best', 'random'],
    'max_depth':[1,2,3,4,5,6,7,8],
    'min_samples_split':[1,2,3],
    'min_samples_leaf':[1,2,3],
    'max_features':['auto', 'sqrt', 'log2']
}
```

```
[23]: params
```

```
[23]: {'criterion': ['gini', 'entropy', 'log_loss'],
      'splitter': ['best', 'random'],
      'max_depth': [1, 2, 3, 4, 5, 6, 7, 8],
      'min_samples_split': [1, 2, 3],
      'min_samples_leaf': [1, 2, 3],
      'max_features': ['auto', 'sqrt', 'log2']}
```

```
[24]: from sklearn.model_selection import GridSearchCV

tree_model = DecisionTreeClassifier()

grid = GridSearchCV(estimator=tree_model, param_grid=params, cv=5,
    ↪scoring='accuracy')
```

```
[25]: grid.fit(X_train, y_train)
```

```
[25]: GridSearchCV(cv=5, estimator=DecisionTreeClassifier(),
    param_grid={'criterion': ['gini', 'entropy', 'log_loss'],
                'max_depth': [1, 2, 3, 4, 5, 6, 7, 8],
                'max_features': ['auto', 'sqrt', 'log2'],
                'min_samples_leaf': [1, 2, 3],
                'min_samples_split': [1, 2, 3],
                'splitter': ['best', 'random']}},
    scoring='accuracy')
```

```
[26]: grid.best_estimator_
```

```
[26]: DecisionTreeClassifier(max_depth=5, max_features='sqrt', min_samples_leaf=3)
```

```
[27]: grid.best_params_
```

```
[27]: {'criterion': 'gini',  
      'max_depth': 5,  
      'max_features': 'sqrt',  
      'min_samples_leaf': 3,  
      'min_samples_split': 2,  
      'splitter': 'best'}
```

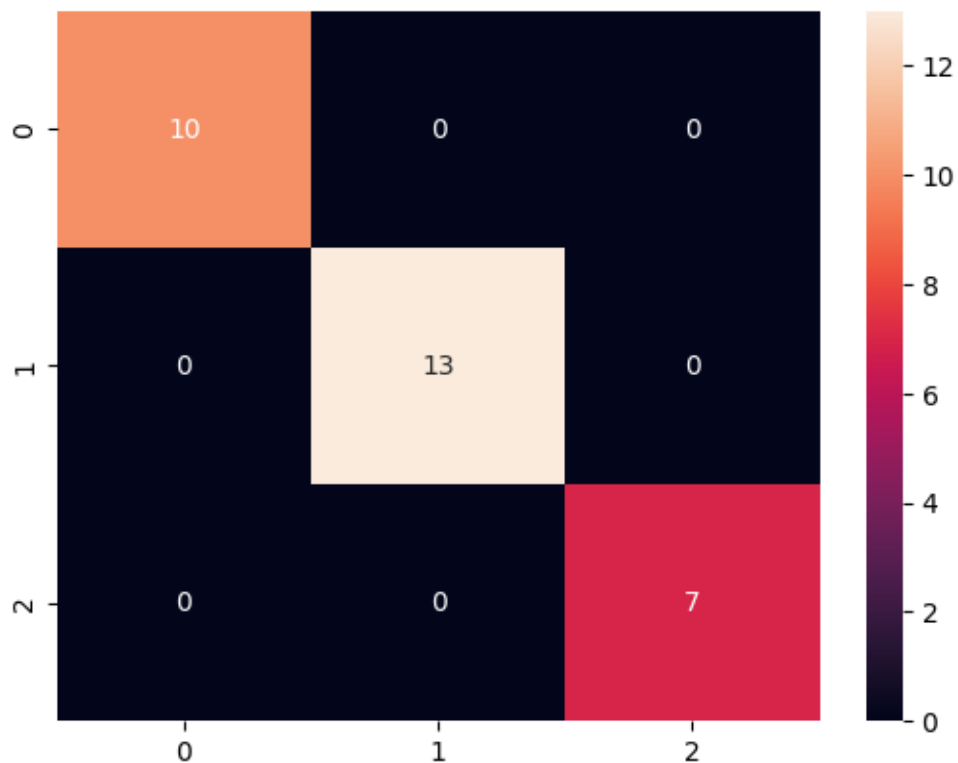
```
[28]: grid.best_score_
```

```
[28]: 0.9583333333333334
```

```
[29]: y_pred = grid.predict(X_test)
```

```
[30]: sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt='d')
```

```
[30]: <Axes: >
```



```
[31]: print(accuracy_score(y_test, y_pred))  
  
print(classification_report(y_test, y_pred))
```

1.0

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	1.00	1.00	13
2	1.00	1.00	1.00	7
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30